

Manual de Healthcheck por Componentes

Ultima actualizacion: 2026-05-01

Proyecto: AI DocClassExt — DocumentIA MVP

Endpoint: POST /api/healthcheck (Function App srbappprodocai)

Codigo: src/backend/DocumentIA.Functions/Services/SystemHealthService.cs + Triggers/HealthcheckFunction.cs

Este manual describe los probes que componen el healthcheck del sistema, su semantica, los settings que controlan cada uno, los consumidores conocidos, las situaciones de fallo tipicas y como diagnosticarlos.

Para el contrato completo de request/response ver docs/contratos/CONTRATO_API_HTTP.md §4.bis.

1. Vision general

Aspecto	Valor
Endpoint	POST /api/healthcheck
Auth	AuthorizationLevel.Anonymous (sin Function Key)
Metodo	POST (no soporta GET)
Response codes	200 OK (healthy/degraded/unconfigured) o 503 Service Unavailable (unhealthy)
Cache interno	45 s en IMemoryCache (clave SystemHealthService:ComponentsHealthSnapshot)
Lifetime DI	AddScoped<ISystemHealthService, SystemHealthService>() (depende de IGdcService que es scoped)
Fallback	Si ISystemHealthService no esta inyectado devuelve payload minimo { ok: true, timestamp: "..." }

2. Estados posibles

Cada componente y el agregado pueden tomar uno de estos cuatro estados (en minusculas):

Status	Significado	Impacto en agregado
healthy	Probe completado con exito.	Si todos son healthy , agregado healthy .
degraded	Probe respondio pero con codigo no satisfactorio o timeout no critico.	Eleva agregado a degraded si no hay unhealthy .
unhealthy	Probe fallo (excepcion, timeout critico).	Eleva agregado a unhealthy y devuelve 503 .
unconfigured	Falta setting obligatorio o el loader/servicio no esta registrado.	Eleva agregado a unconfigured si no hay degraded ni unhealthy .

Precedencia agregada: unhealthy > degraded > unconfigured > healthy .

3. Componentes

3.1 functions

Campo	Valor
Que comprueba	El runtime Functions esta vivo lo suficiente para responder.
Implementacion	Constante: <code>ComponentHealth.Healthy("Running")</code> . No ejecuta probe externo.
Falla cuando	Nunca (si fallara, no se serviria la respuesta).
Settings relevantes	Ninguno especifico.
Como diagnosticar fallo	Si el endpoint no responde, el problema es de la Function App: revisar <code>srbappprodoci</code> en el portal, App Insights <code>srbappprodoci</code> y Resource Health.

3.2 assetResolver

Campo	Valor
Que comprueba	Que el Web App del plugin AssetResolver responde a <code>GET /api/assets/ping</code> .
Implementacion	<code>IHttpClientFactory.CreateClient("AssetResolver").GetAsync("api/assets/ping")</code> con timeout 8 s.
Settings	<code>AssetResolver:BaseUrl</code> y <code>AssetResolver:ApiKey</code> (en KV via <code>@Microsoft.KeyVault(...)</code> en prod). El cliente HTTP nombrado se configura en <code>Program.cs</code> con base URL y header de API key.
healthy	Respuesta 2xx. <code>message: "HTTP 200"</code> .
degraded	Respuesta no-2xx. <code>message: "HTTP {code}"</code> .
unhealthy	Excepcion HTTP o timeout > 8 s. <code>message: "Timeout"</code> o mensaje de excepcion.
unconfigured	<code>AssetResolver:BaseUrl</code> o <code>AssetResolver:ApiKey</code> vacios.
Diagnostico	1) Verificar Web App <code>srbwebpluginassetresolver</code> esta arrancado y responde a <code>GET /api/assets/ping</code> . 2) Si KV reference, validar Managed Identity tiene <code>Key Vault Secrets User</code> sobre <code>srbkvprodoci</code> . 3) Revisar logs Plugin Web App.

3.3 gdc

Campo	Valor
Que comprueba	Que el endpoint SOAP de GDC responde a <code>consultarObjetoExiste</code> .
Implementacion	<code>IGdcService.ConsultarDocumentoAsync("HEALTHCHECK_PROBE", "00000000000000000000000000000000", "HLTH", ct)</code> con timeout 5 s.
Settings	<code>GDC:Endpoint</code> (URL SOAP), <code>GDC:Usuario</code> , <code>GDC:Password</code> , <code>GDC:Aplicacion</code> .
healthy	Llamada completada (devuelva <code>exists=true</code> o <code>false</code>). <code>message: "Reachable (document found)"</code> o <code>"Reachable (document not found)"</code> .
degraded	Timeout > 5 s (no se considera unhealthy porque GDC ocasionalmente responde lento sin error funcional).
unhealthy	SOAP fault, error de red, credenciales rechazadas, etc.
unconfigured	<code>GDC:Endpoint</code> vacio.
Diagnostico	1) Comprobar conectividad de red al endpoint SOAP (puede requerir whitelist NSG). 2) Validar credenciales en KV (<code>gdc-usuario</code> , <code>gdc-password</code>). 3) Probar con <code>tests/api-tests/test-gdc-consultar-aislado.ps1</code> .

3.4 modelProviders

Estructura compuesta con tres sub-componentes (cada uno objeto unico, no array):

3.4.1 modelProviders.classification

Campo	Valor
Que comprueba	Que <code>ClassificationModelRegistryLoader</code> esta registrado en DI.
<code>healthy</code>	Loader presente. <code>message: "Loader registered"</code> .
<code>unconfigured</code>	Loader no registrado (configuracion DI defectuosa).
Settings	Implicitos: la presencia del loader implica que se cargo el registry de modelos de clasificacion.
Diagnostico	Revisar <code>Program.cs</code> y la configuracion de modelos en BBDD (<code>ai-models/Classification</code>).

3.4.2 modelProviders.extraction

Identico al anterior pero con `ExtractionModelRegistryLoader` .

3.4.3 modelProviders.prompt

Identico al anterior pero con `PromptModelRegistryLoader` .

Limitacion conocida: el probe actual no valida la conectividad real con cada modelo (DI / Azure OpenAI / Foundry). Solo confirma que el loader estatico esta cargado. La conectividad real con cada modelo se ejercita al usarlo en runtime (clasificar / extraer / prompt).
Mejora candidata: ejecutar un ping ligero por proveedor.

4. Cache de 45 segundos

El snapshot se cachea para evitar martillear servicios externos (GDC, AssetResolver) cuando hay multiples consumidores polleando.

- TTL: 45 s.
- Clave: `SystemHealthService:ComponentsHealthSnapshot` .
- Implementacion: `IMemoryCache` (proceso). Cada instancia de Function host tiene su propia cache.
- Implicacion: si un componente cae, el primer probe tras la ventana lo detecta; los probes siguientes (en los 45 s posteriores) devuelven el mismo resultado en cache.

5. Consumidores

5.1 Frontend Admin (`DocumentIA.Admin` , Blazor)

- Servicio: `MonitorService.GetSystemHealthAsync()` .
- Llamada: `POST /api/healthcheck` .
- Comportamiento: acepta tanto `200` como `503` con payload JSON. Pinta tarjeta "Salud del sistema" en pagina `Monitor` con sub-cards por componente.
- Polling: bajo demanda (refresco manual o navegacion a Monitor).

5.2 Frontend Desktop (`DocumentIA.Desktop` , WPF)

- Cliente: `OrchestratorApiClient.GetSystemHealthAsync()` .
- ViewModel: `ProcessingViewModel` expone bindings de salud agregada y por componente.
- UI: panel AGG / FUNCTIONS / ASSET / GDC / MODELS / UPDATED en `MainWindow.xaml` .
- Conversor: `HealthStatusToBrushConverter` en `Styles.xaml` traduce status a color.

5.3 Tests

- `DocumentIA.Tests.Unit.Triggers.HealthcheckFunctionTests` (8 tests): cubre casos sin servicio (fallback minimo), con servicio healthy, degraded, unhealthy, unconfigured y verifica codigo HTTP.

5.4 Probes externos (recomendado)

Consumidor	Configuracion sugerida
Application Insights Availability Test	URL POST <code>https://srbappprodoci.azurewebsites.net/api/healthcheck</code> , esperar <code>200</code> , frecuencia 5 min, regiones multiples.
Azure Front Door / balanceador (futuro)	Health probe HTTP POST <code>/api/healthcheck</code> , intervalo 30 s, umbral 3 fallos consecutivos antes de marcar instance unhealthy.
Workbook App Insights	KQL sobre <code>requests</code> filtrando <code>name == "PostHealthcheck"</code> para latencia y tasa de error.

6. Pruebas locales

```
# Funcion local con func host start
$body = @{} | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri "http://localhost:7071/api/healthcheck" -Body $body -ContentType "application/json"
```

```
# Productivo
Invoke-RestMethod -Method Post -Uri "https://srbappprodoci.azurewebsites.net/api/healthcheck"
```

Si solo se quiere comprobar el endpoint sin probes (escenario tests), puede inyectarse un host alternativo sin `ISystemHealthService` registrado: la respuesta sera el payload minimo `{ ok: true, timestamp: "..." }`.

7. Troubleshooting

Sintoma	Probable causa	Accion
<code>503</code> con <code>assetResolver.status == "unhealthy"</code> y <code>message: "Timeout"</code>	Web App AssetResolver caido o cold-start.	Revisar Resource Health del Web App; forzar warm-up; revisar logs.
<code>assetResolver.status == "unconfigured"</code> en prod	KV reference no se resolvio.	Verificar Managed Identity tiene rol <code>Key Vault Secrets User</code> en <code>srbkvprodoci</code> ; reiniciar Function App; comprobar <code>local.settings.json</code> no override.
<code>gdc.status == "degraded"</code> con <code>Timeout</code>	Latencia GDC alta pero servicio funcional.	Monitorizar; si persiste subir alerta. No bloquea el sistema.
<code>gdc.status == "unhealthy"</code> con SOAP fault	Credenciales rechazadas o endpoint cambio.	Validar <code>gdc-usuario</code> / <code>gdc-password</code> en KV; ejecutar <code>test-gdc-consultar-aislado.ps1</code> .
<code>modelProviders.*.status == "unconfigured"</code>	Loader no registrado en DI.	Bug de configuracion: revisar <code>Program.cs</code> y verificar que el registry de modelos esta presente.
Endpoint devuelve <code>200</code> con payload minimo <code>{ ok: true, timestamp }</code>	<code>ISystemHealthService</code> no se inyecto (constructor sin parametros).	Reiniciar Function App; verificar <code>AddScoped<ISystemHealthService, SystemHealthService>()</code> en <code>Program.cs</code> .
Endpoint devuelve <code>404</code>	Despliegue antiguo sin la function.	Republicar Functions (<code>func azure functionapp publish srbappprodoci</code>).

8. Referencias

- `docs/contratos/CONTRATO_API_HTTP.md` §4.bis Healthcheck (request/response oficial).
- `src/backend/DocumentIA.Functions/Services/SystemHealthService.cs` (logica de probes).
- `src/backend/DocumentIA.Functions/Triggers/HealthcheckFunction.cs` (HTTP trigger).
- `src/backend/DocumentIA.Tests.Unit/Triggers/HealthcheckFunctionTests.cs` (cobertura).
- `docs/01_ARQUITECTURA_SISTEMA.md` (modulo healthcheck en diagrama).